

BIG DATA PLATFORMS DATA140031 (MOOC, 3 ECTS) + DATA140032 (MOOC EXAM, 2 ECTS

LECTURE 11

Keijo Heljanko and Ilari Maarala

EXAM REQUIREMENTS

Exam requirements:

- Book an Exam between 19.10-9.12.2026
- See course homepage for Exam registration link

- Lectures 1-10 are part of the Exam requirements
- The Home Assignments are part of the Exam requirements
- The rest of this Lecture 11 is not part of the Exam requirements



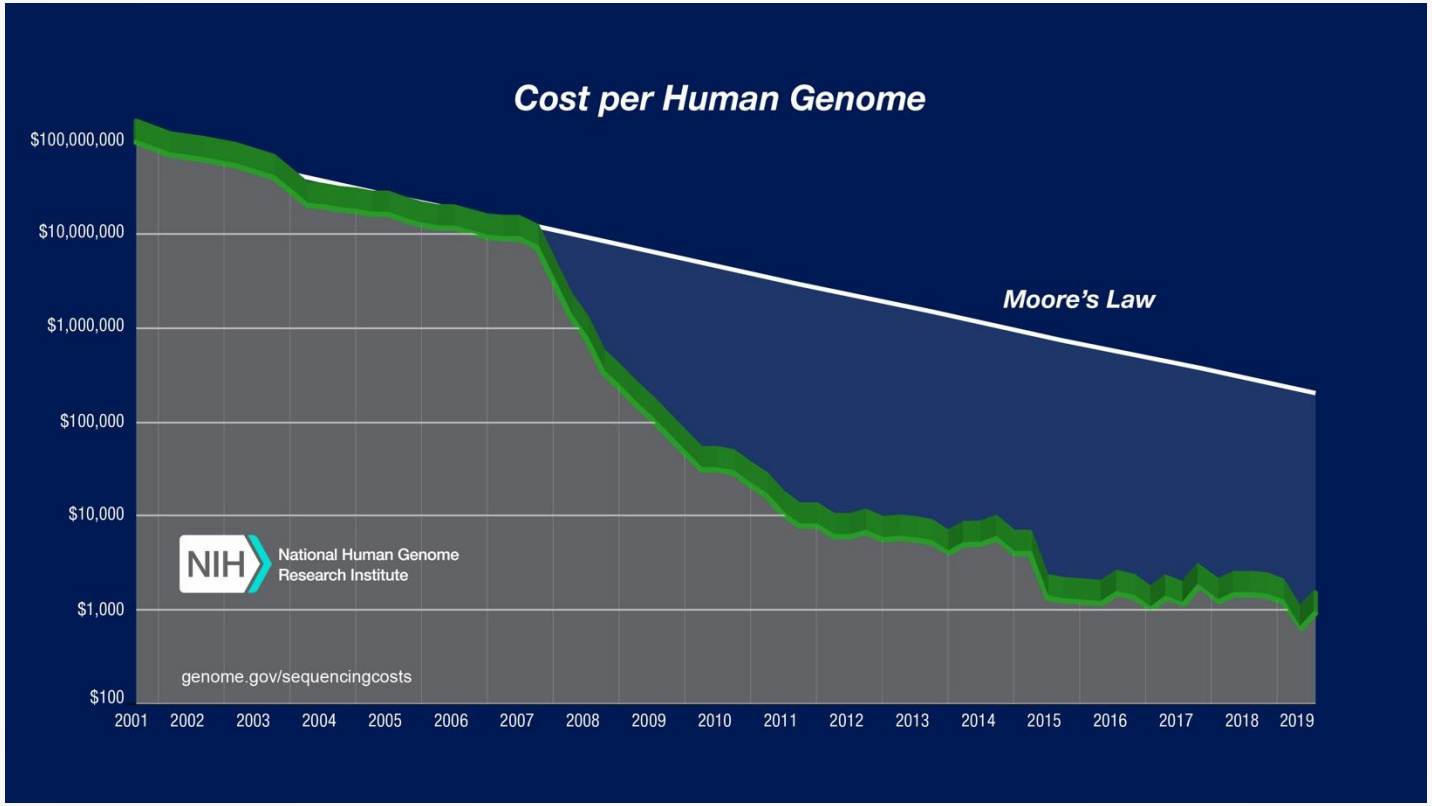
USING SPARK FOR DISTRIBUTED PROCESSING OF GENOMICS DATA

- In this Lecture we show a couple of case studies of the use of Spark to process genomics data

INTRODUCTION

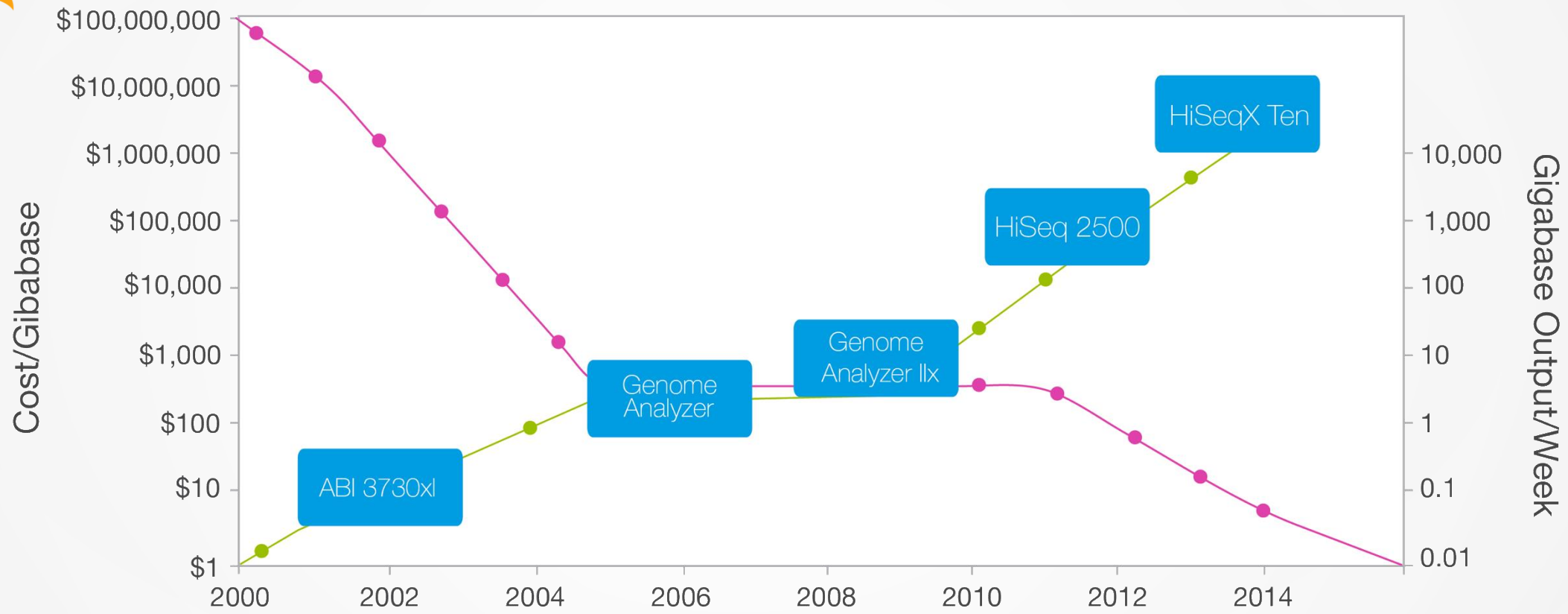
- The amount of DNA sequencing data worldwide has been growing at high rate (>> Moore's law). The race is on between the sequencing data throughput and computing speed.
- Petabytes of Next-generation sequencing (NGS) data sets are in public repositories (NCBI, EMBL-EBI, 1000 Genomes).
- NGS data has typically been processed with traditional sequential algorithms or by using shared memory parallel computing on a single machine.
- NGS analytics methods have to cope with this data growth rate.
- Towards distributed computing methods and algorithms to avoid hitting computational limits and the "bioinformatics bottleneck".

INTRODUCTION



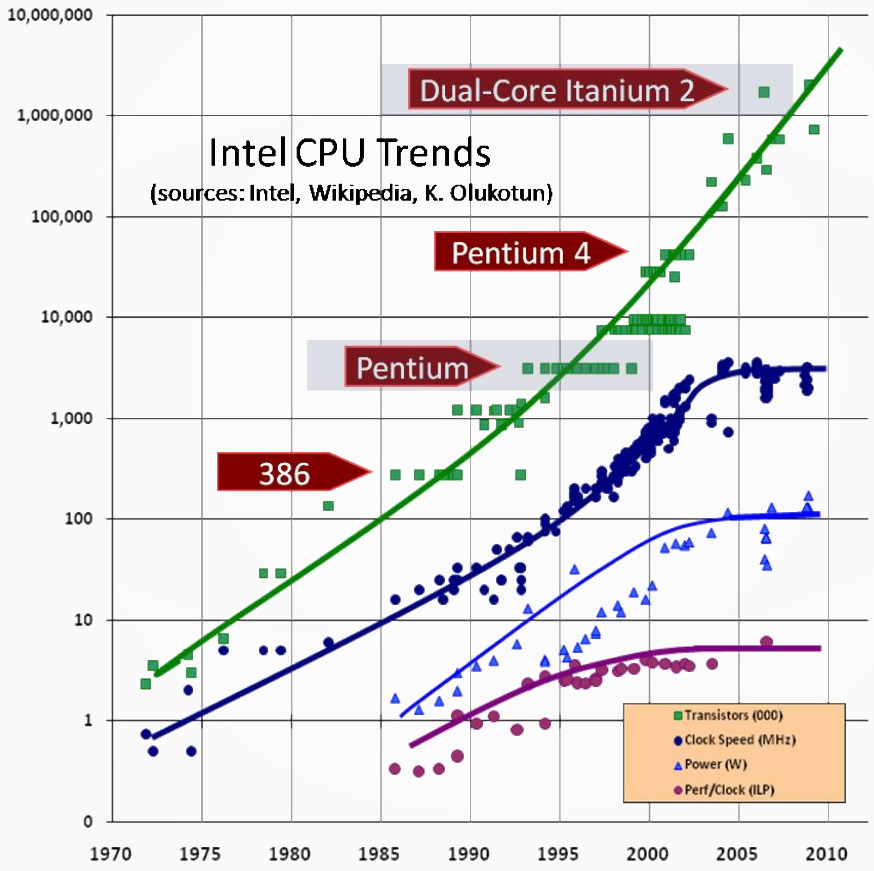
Approximate cost of sequencing a human genome since 2001. Image: National Human Genome Research Institute (NHGRI) - https://www.genome.gov/sequencingcosts/

COST OF NGS DATA GENERATION



Next-generation sequencing data output rate versus sequencing cost on Illumina NGS platforms. (Image source: An introduction to Next-Generation Sequencing Technology, Illumina Inc.)

SEQUENTIAL SPEED STAGNATION



Herb Sutter: The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. Dr. Dobb's Journal, 30(3), March 2005 (updated graph in August 2009).



SCALABLE REFERENCE GENOME ASSEMBLY FROM COMPRESSED PAN-GENOME INDEX WITH SPARK

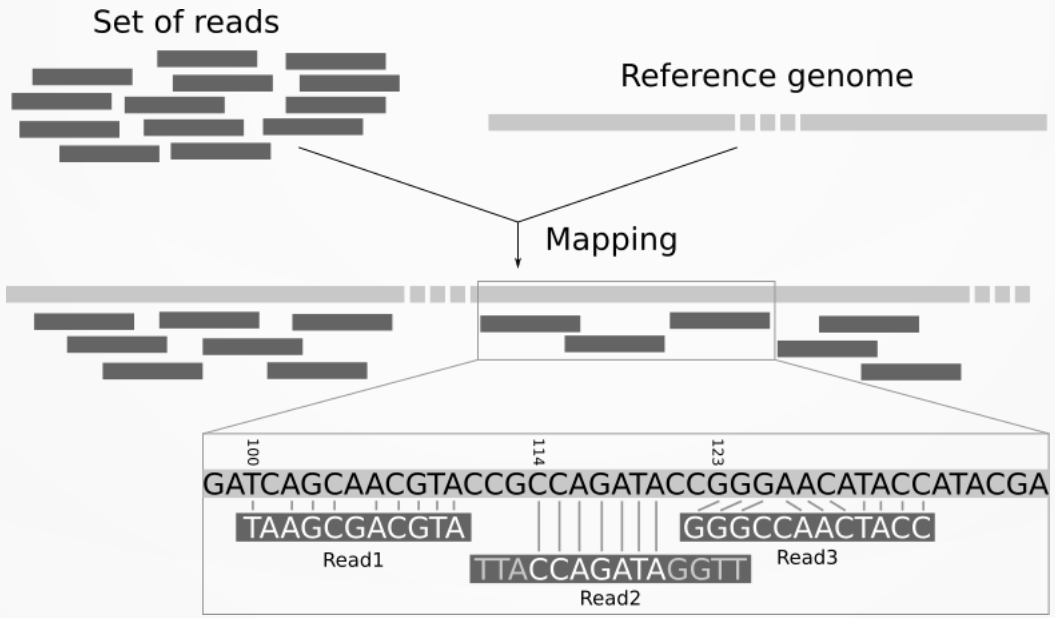
Altti Ilari Maarala, Ossi Arasalo, Daniel Valenzuela, Keijo Heljanko, Veli Mäkinen

PAN-GENOMICS

- The first human reference genome draft assembled in 2004 by the Human Genome Project and its successors are used as a comparative reference in the majority of human genetic studies.
- Recent pan-genomic studies have revealed large amounts of novel genetic material that is not included in the standard human reference genome
- **Pan-genome** can include **genomes of the whole population** and represent diverse populations without disabling any population- or individual-specific genetic variation => Improves the genetic variation analysis resolution => Considers the genetic recombination and emphasizes the diversity of individuals.
- Constructing a reusable reference genome for sequence searching and aligment requires indexing of all genomes in a population => Huge computational burden.

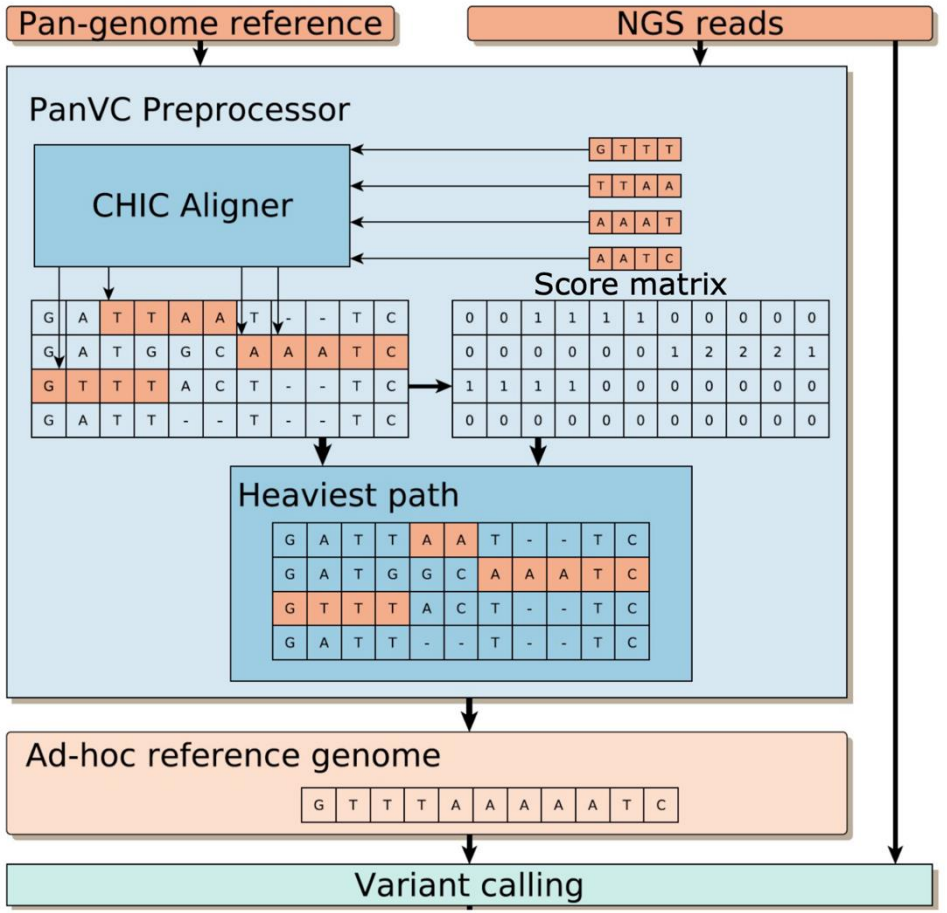
ASSEMBLING A REFERENCE GENOME

- A DNA sample is sequenced in short fragments (reads) with NGS => Reads **are aligned to a reference genome**.
- See: https://training.galaxyproject.org/training-material/topics/sequence-analysis/tutorials/mapping/tutorial.html



PAN-GENOMIC REFERENCE

- Pan-genomic reference is a multiple sequence alignment of N genomes => Reads **are aligned to all genomes** in a pan-genome.

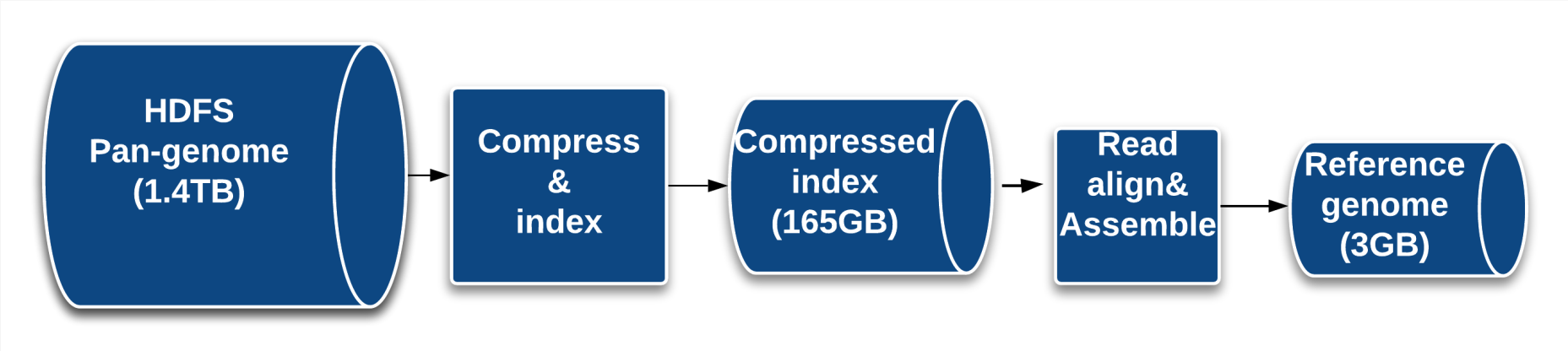


COMPRESSING PAN-GENOME

- Genomes are highly repetitive sequences compared across indivuals => Pan-genome can be compressed efficiently.
- Hybrid stategy: Compressed pan-genome is indexed => Reads aligned (mapped) using a compressed index of the pan-genome.
- The best mapping reads across the pan-genomic positions (”majority vote”) form the new **consensus reference genome for the population**.

PAN-GENOME DATA VOLUMES

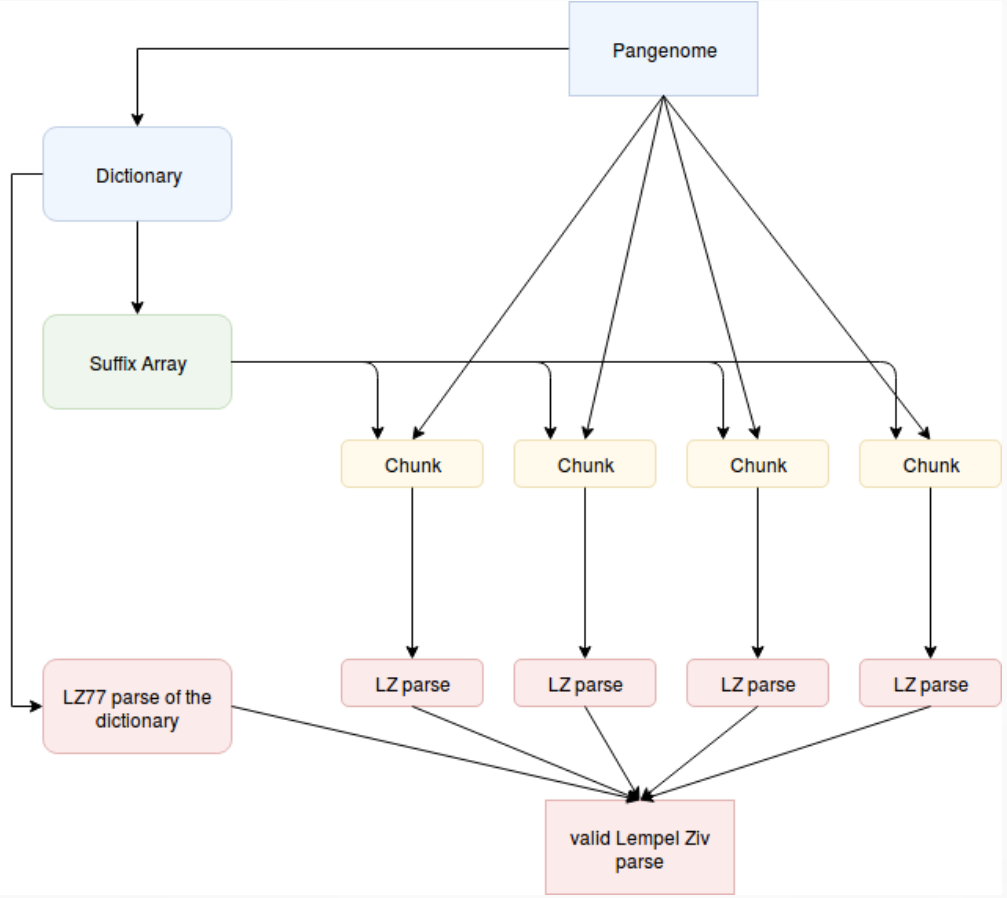
- Pan-genome is represented using in a two-stage compressed hybrid index:
  - First compressed with Lempel-Ziv based compression into a compressed Lempel-Ziv parse
  - Lempel-Ziv parse is indexed using Burrows-Wheeler based compressed full-text search index
- Consensus reference genome is the ”majority vote” for the population in the database



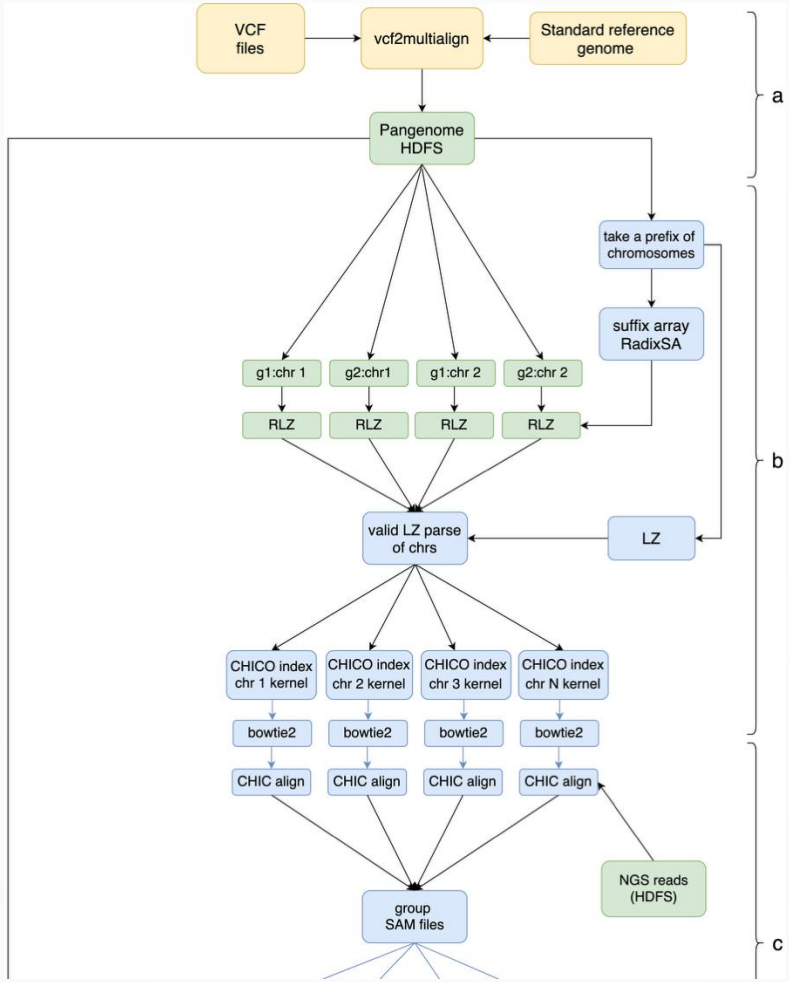
OUR APPROACH

- We focus on indexing and compressing reusable pan-genomic reference assemblies emphasizing speedup, scalability, and space-efficiency.
- Our tool, PanGenSpark is a parallelization of a sequential pan-genomic variation calling pipeline PanVC and CHIC indexer (Valenzuela et al.).
- PanGenSpark parallelizes the most compute-intensive phases in PanVC such as compressing and indexing of large pan-genomes and assembling the consensus genome from a pan-genome.

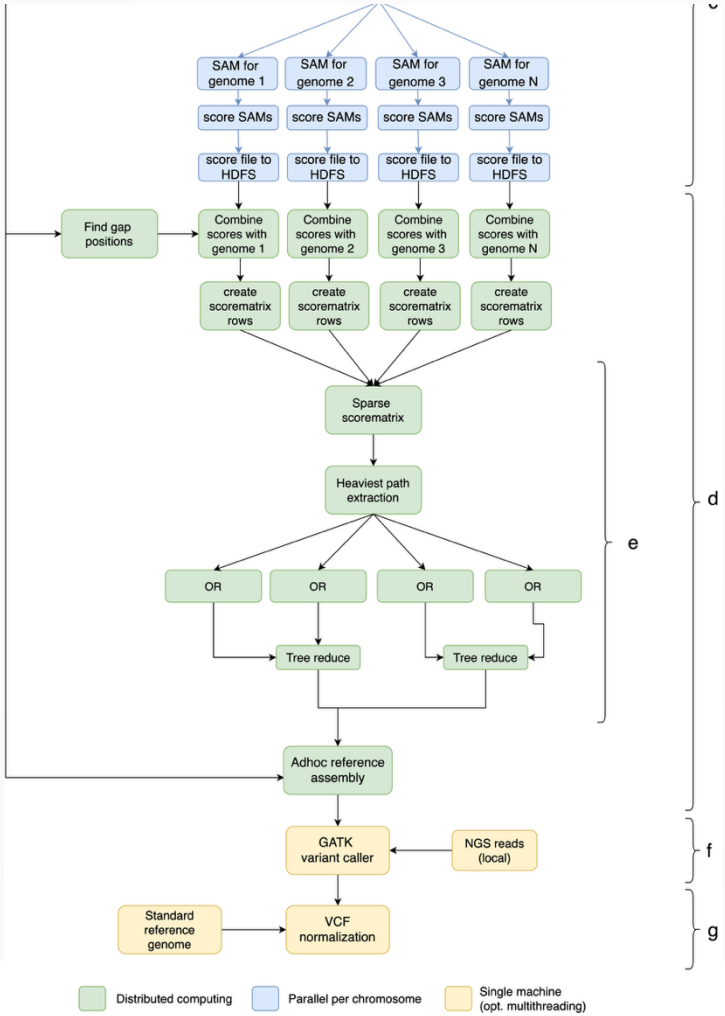
DISTRIBUTED RELATIVE LEMPEL-ZIV PARSING (COMPRESSION)



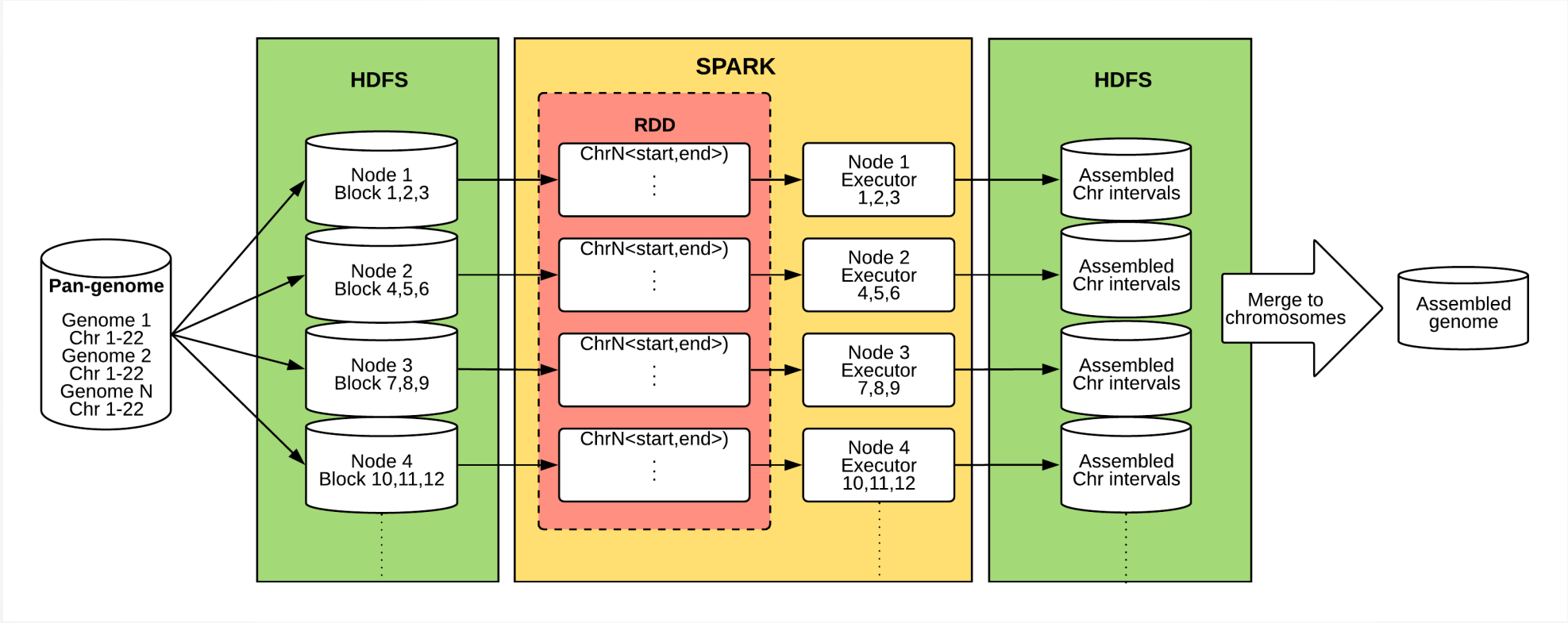
DISTRIBUTED ASSEMBLY WITH PANGENSPARK



DISTRIBUTED ASSEMBLY WITH PANGENSPARK



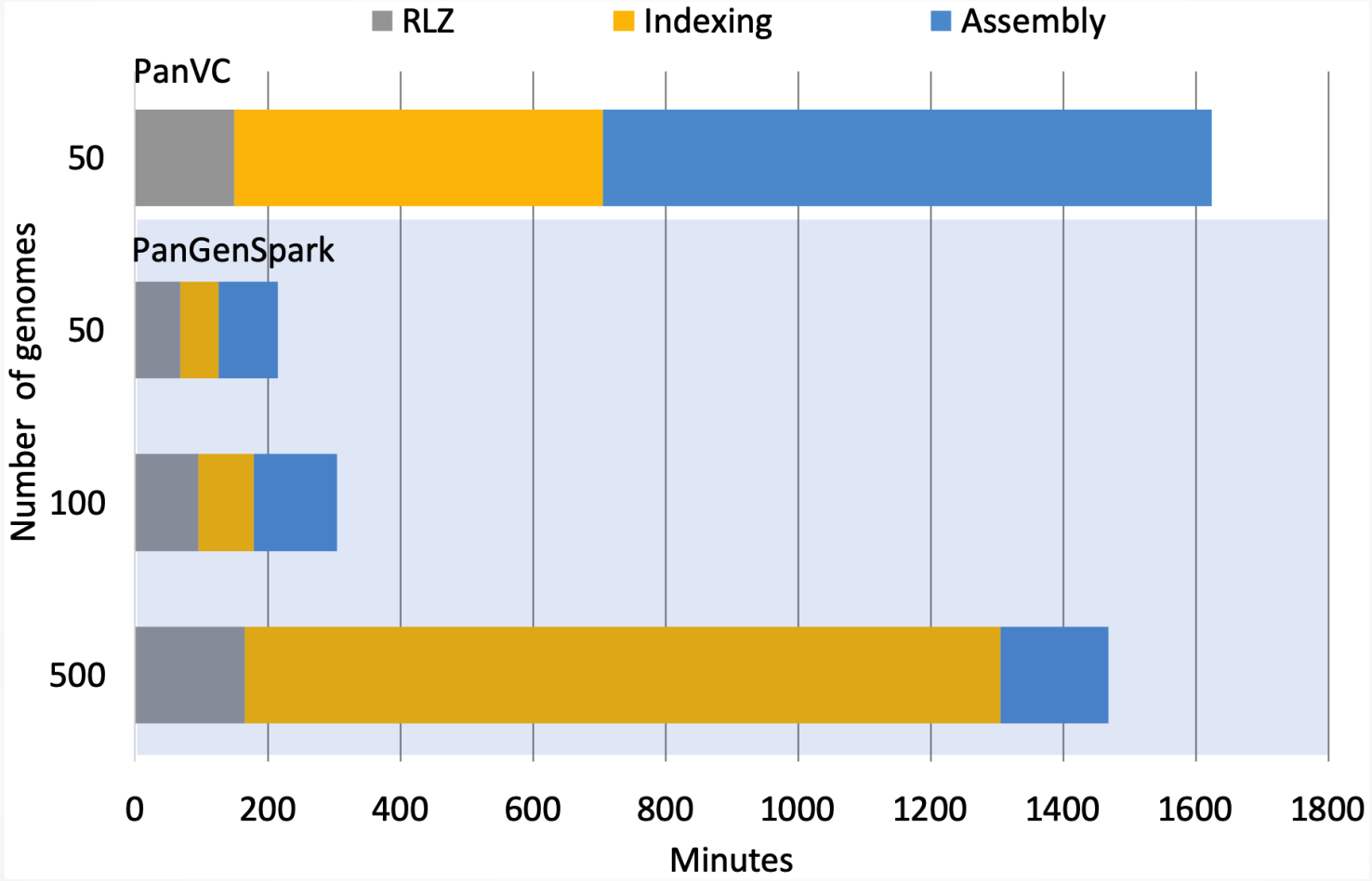
SPARK+HDFS STACK



EXPERIMENTS

- The scalability experiments conducted on the Apache Spark cluster in a dedicated cloud environment.
- The number of individual genomes in a pan-genome is increased. Wallclock time from compression to assembly stage is measured. The results are compared between the PanVC and PanGenSpark.
- 448 CPU cores, 1.256 TB of RAM, Infiniband 40 GB/s network, 30 TB HDFS storage in total.
- 25 Spark worker nodes: 40 GB of RAM and 16 cores (Intel(R) Xeon(R) CPU E5-2680 v3) in each + 1 Spark master node: 256 GB of RAM and 48 cores.
- Apache Spark 2.3.2 and Hadoop 3.1.0 on virtual machines running CentOS 7.

SCALABILITY RESULTS



COMPRESSION RESULTS

Genomes (N)	Uncompressed (GB)	Compressed (GB)	CR	
1	2.82	0.8	3.53	
50	141	24.7	5.71	
100	282	36.4	7.74	
500	1410	164.5	8.57	
Preliminary results with 100 Genomes using longer LZ dictionary				
Dict.size (N)	DRLZ (min)	Compressed (GB)	CR	bpc
1	91.5	36.4	7.39	0.135
8	191	6.3	42.7	0.023
15	236.5	3.3	81.52	0.012

SUMMARY

- Compressing reusable human pan-genomic ”legacy” indexes and assembling reference genomes in a tolerable time is now possible in practice.
- Indexing stage with ”legacy” indexers is a bottleneck (BWA limited to 4 gigabases, Bowtie2 not limited but not scaling linearly).
- Limiting issue to use whole genome as a dictionary with Spark is the sequence length > Java’s 32bit integer arrays < human genome (3 gigabases).
- Minimizing I/O operations with large data sets in a complex pipeline. Input data structure varies in different stages => Complex data and task partitioning challenges distributed pipeline and algorithm development.

PANGENSPARK

PanGenSpark is available in https://github.com/NGSeq/PanGenSpark

Contact: ilari.maarala@helsinki.fi



SparkBeagle: Scalable Genotype Imputation from Distributed Whole-Genome Reference Panels in the Cloud

Altti Ilari Maarala, Kalle Pärn, Javier Nuñez-Fontarnau, Keijo Heljanko

ParBio-2020
ACM-BCB

INTRODUCTION

- Genotyping is routine method for detecting genomic variation (SNPs). Genotype imputation from large Whole Genome Sequencing(WGS) -based panels increase resolution and precision in GWAS.
- The basic idea is to use to use cheap analysis techniques to measure some SNPs, and use statistics over the population to statistically infer the missing SNPs
- Sequential imputation algorithms introduce computational challenges.
- SparkBeagle = Distributed implementation of popular Hidden markov model based Beagle statistical imputation software built on the Apache Spark and Hadoop Distributed Filesystem = > Scalability, speedup, and fault tolerance.
- SparkBeagle can be deployed on cloud based computing clusters.

IMPUTATION SCENARIO

Marker	1	2	3	4	5	6	7	8
	1	0	1	1	0	0	1	0
	0	0	0	1	0	0	0	1
	0	0	1	1	0	1	1	0
	0	1	1	0	0	0	1	0
	1	0	1	1	0	0	0	0
	1	0	1	1	1	0	1	0
	0	0	1	0	1	0	1	1
	1	1	0	0	0	0	1	0
	0	?=2	1	2	0	?	?	1
	0	?	2	?=2	2	0	?=2	1
	1	2	1	?	1	?	?	0
	1	?	1	0	1	?	?	1
	?	2	2	?	0	1	?	0
	1	?=0	1	?=1	0	?	?	0

> 81M SNP markers in 1000G panel

Reference haplotypes 5008x

Target genotypes (low density)

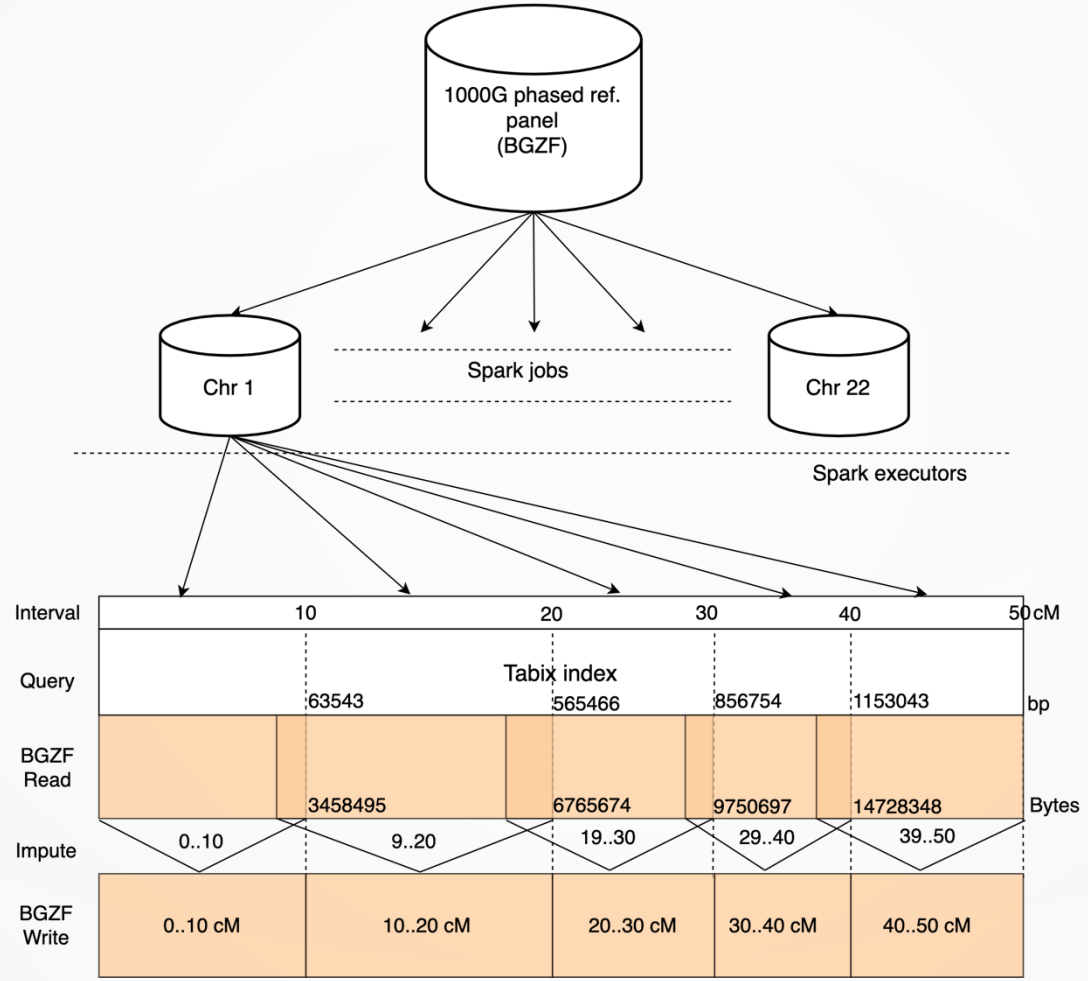
Interval 1 (Chr N:1-5) | overlap | Interval 2 (Chr N:4-8)

Executor 1 Impute Interval 1

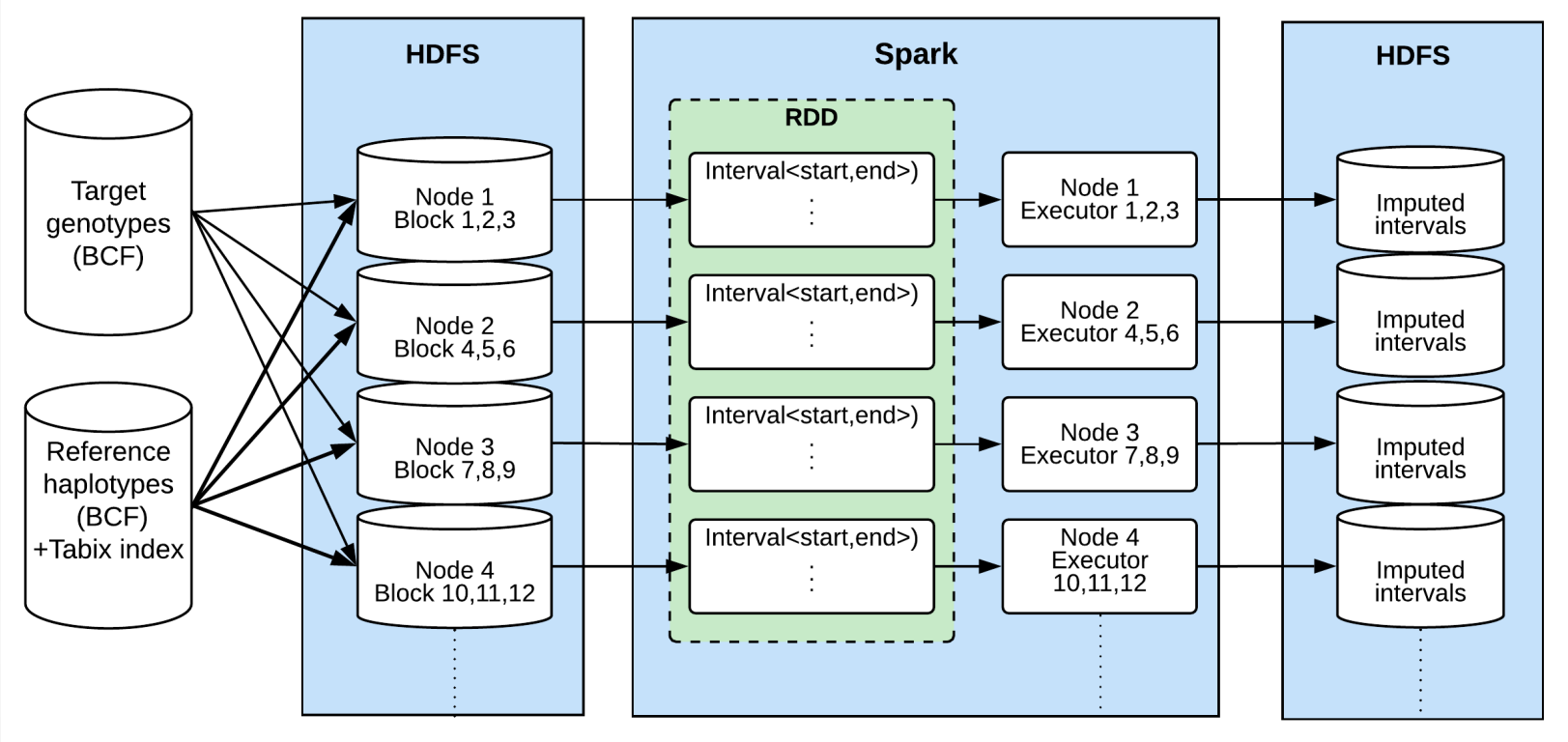
Executor 2 Impute Interval 2

missing target genotype = ?
homozygous major allele = 0
homozygous minor allele = 1
heterozygous allele = 2

IMPLEMENTATION



IMPUTATION PIPELINE





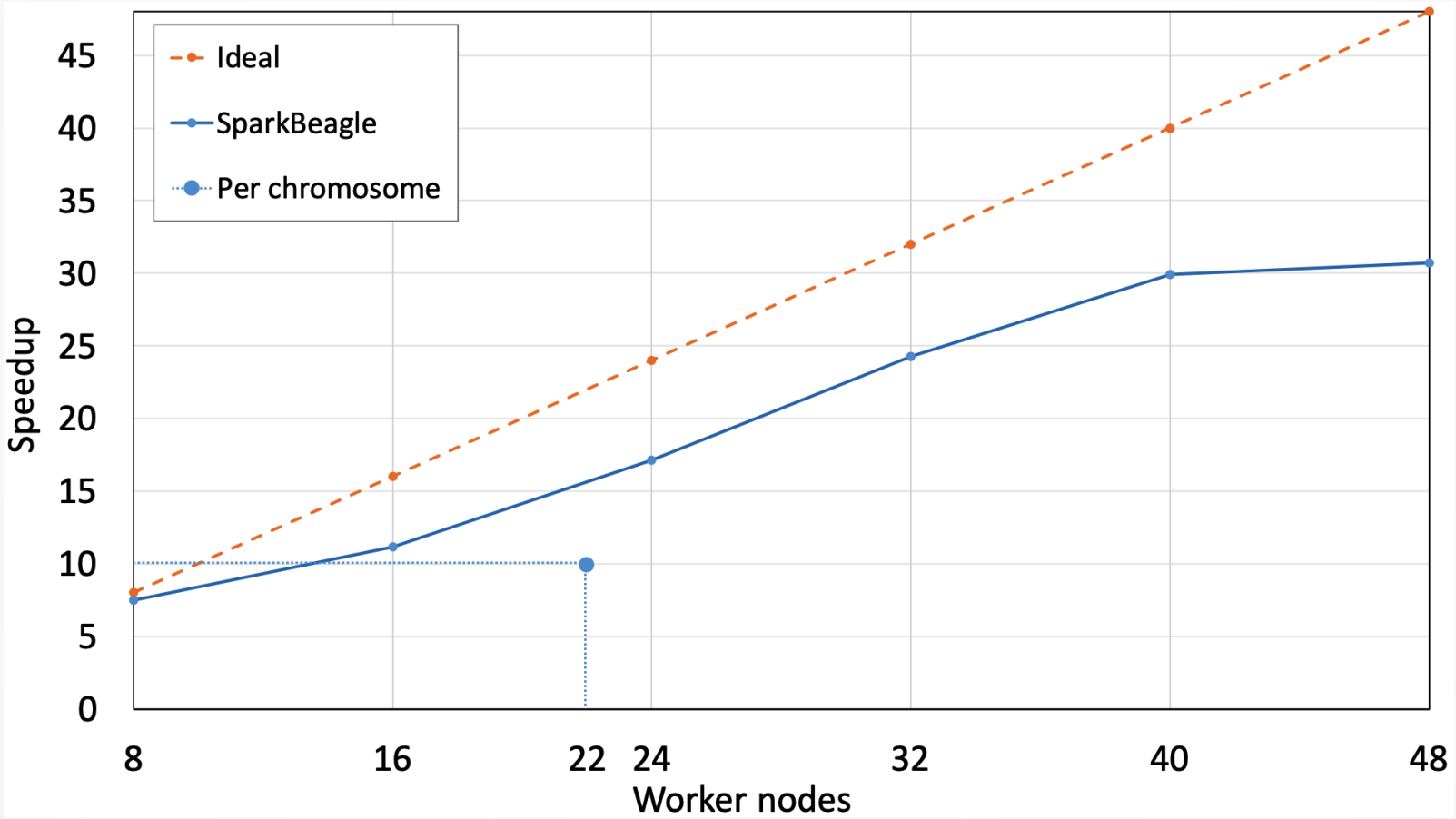
CLUSTER SETUP

- The experiments are conducted on the Apache Spark cluster in a dedicated cloud environment.
- 520 CPU cores, 1.15 TB of RAM, Infiniband 25 GB/s network, 15 TB HDD storage for HDFS.
- 48 Spark worker nodes: 44 GB of RAM and 10 cores (Intel Xeon Gold 6148, with hyper-threading) in each. Two Spark master nodes: 86 GB of RAM and 20 cores in each.
- Deployed with Hortonworks Data Platform (HDP) 3.1 distribution on virtual machines running CentOS 7.
- Spark 2.3.2 and Hadoop 3.1.0 with Yarn resource manager.
- Job queue fair ordering policy and size based weighting => Physically larger imputation intervals have more resources => better load balancing.

SCALABILITY EXPERIMENT

- 1000 Genomes phase 3 data as reference panel: 5008 haplotypes (81,214,785 SNP markers, 769 GB, chromosomes 1-22).
- 1/5 SNPs filtered for the target data (16,238,469 markers, 154 GB) => ~65M markers imputed over 2500 times!
- The dataset size is kept constant, the cluster size is increased from 8 to 48 nodes (80 to 480 cores)
- As a reference, the same data is imputed also with Beagle distributed per chromosome to separate nodes (typical scenario with workload managers like Slurm).

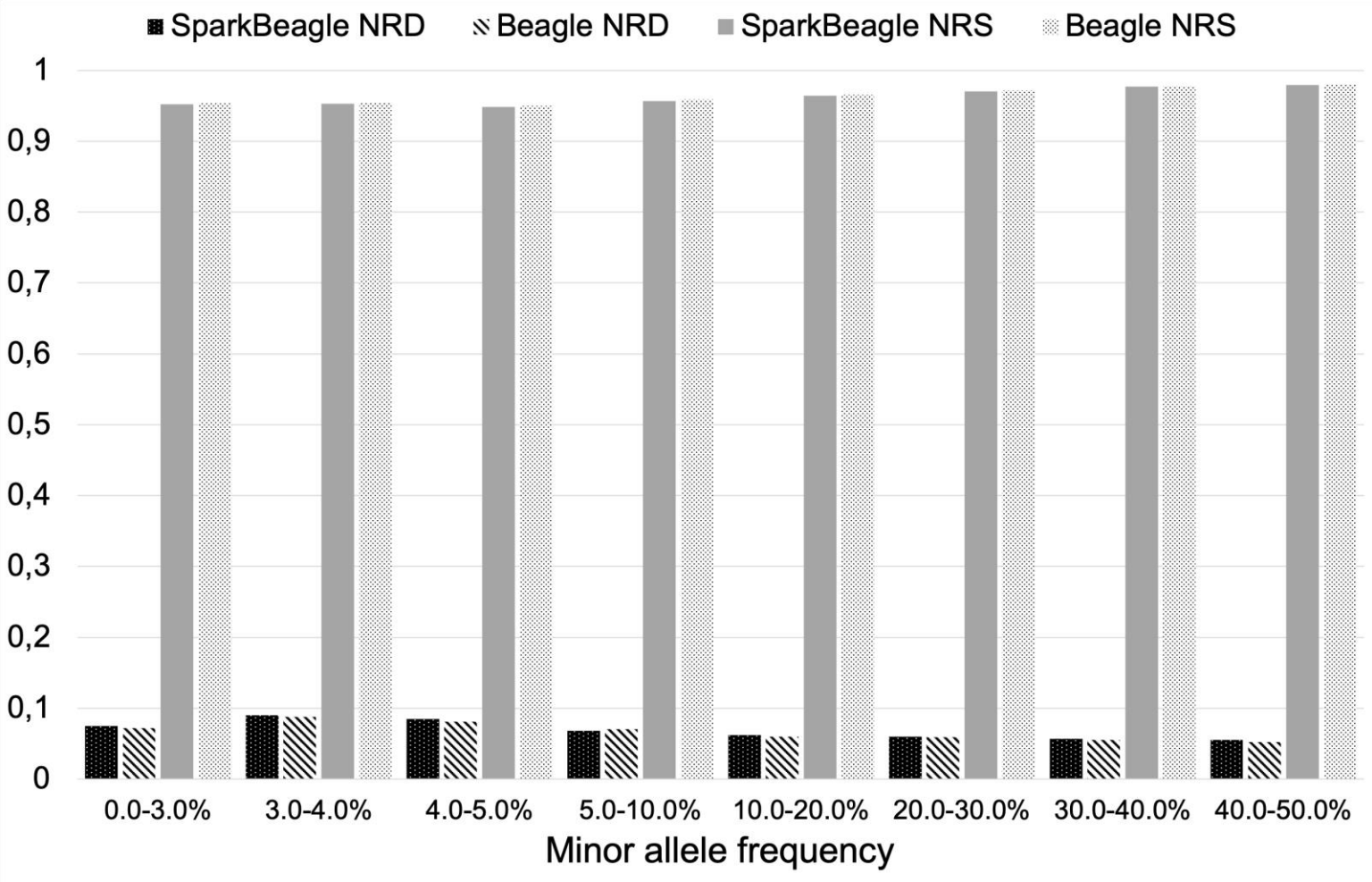
SCALABILITY RESULTS



10 cores/node
480 cores in total
365 intervals in total
1 core/interval

SparkBeagle
Imputation time:
18 minutes with 40 nodes

ACCURACY RESULTS



Non-reference sensitivity (NRS) measures the proportion of recovered genotypes compared to the reference data set and non-reference discrepancy (NRD) measures the proportion of incorrectly imputed genotypes across MAF bins.

SUMMARY

- Genotype imputation is cost-efficient versus microchip-based genotyping.
- Whole Genome Sequencing (WGS) => Huge high density reference panels = > More accurate, fast and lower cost genotyping => more comprehensive GWA studies.
- Widely used imputation tools, complex algorithms, and genomic file formats are not developed for distributed systems.
- Problem and data decomposition of genomics data structures (i.e. by chromosomes, locus, genes, samples) enables data parallel computing and distributed processing.
- SparkBeagle reached 30x speedup on 40 node cluster (imputation time reduced from 565 to 18 minutes). Near identical imputation accuracy with Beagle. Scalability can be improved using shorter intervals => higher parallelization.
- Data parallel methods applicable to other imputation tools if based on chromosomal regions.

SPARKBEAGLE

We would like to thank the Nordic Information for Action eScience Center (NIASC) and Foundations of Computational Health (FCHealth) program of Helsinki Institute for Information Technology.

SparkBeagle is available in https://github.com/NGSeq/SparkBeagle

Contact: ilari.maarala@helsinki.fi



SPARK IN NGS - SUMMARY

- Spark is well suited for processing NGS data
- Legacy fileformats have not been designed for parallelization – need an easy way to split e.g., compressed NGS fileformats – Hadoop-BAM and DisQ-Bio libraries can help
- Often Spark does just the high level coordination, C/C++ based sequential NGS codes are parallelized using Spark based IO, scheduling, and load balancing
- In many applications Spark based shuffle is used to combine the outputs to a single merged (often sorted) final output
- Some approaches such as the Pan-genome index really require massively parallel computing to become feasible
- **Load balancing** is typically the main bottleneck in **scaling up** to high core counts